Épreuve pratique BAC NSI Sujet 4 :

Gestion des ventes d'un Camping municipal

Cette situation d'évaluation comporte ce document ainsi qu'un fichier de codes dans lequel les données concernant le séjour d'un client est stocké dans un dictionnaire.
Le candidat doit restituer ce document avant de sortir de la salle d'examen. Le candidat doit agir en autonomie et faire preuve d'initiative tout au long de l'épreuve.
En cas de difficulté, le candidat peut solliciter l'examinateur afin de lui permettre de continuer la tâche. Des moments privilégiés pour solliciter l'examinateur sont indiqués dans le document sous la forme d'appels professeur.
L'examinateur peut intervenir à tout moment, s'il le juge utile.

L'objectif est de faciliter la gestion du système de gestion d'un camping municipal.

Les données concernant les clients sont stockés dans un dictionnaire :

```
corr_E = {'nom' : 'CORR','prenom':'Edouard','nb_personne':4,
          'duree_sejour':12,'tarif_journalier':30}
```

On souhaite écrire une fonction qui renvoie le montant dû par un client pour un tarif qui dépend de l'emplacement et pour la durée de séjour (en jour).

Sachant qu'au tarif journalier (en euro) de location de l'emplacement il faut ajouter une taxe de séjour de 2,20 € par jour et par personne.
Exemple de calcul du montant à régler pour un client ayant réservé pour 4 personnes pendant 12 jours un emplacement à 30 € la journée : 30 * 12 + 4 * 2.20 * 12 = 465.6

```
>>> montant_a_regler(corr_E)
465.6
```

Question 1 - Compléter la ligne 5 de la fonction : `montant_a_regler`

```
def montant_a_regler(client):
    """ renvoie le montant en euros
    à régler pour cette réservation d'un client au format dictionnaire"""
    pass
```

Appel 1	**Appeler le professeur pour lui présenter votre fonction et son fonctionnement ou en cas de difficultés.**

Chaque facture doit posséder ce que l'on appelle communément un numéro de facture unique.

En réalité il s'agit d'une chaîne de caractères. Pour ses factures, depuis 2018, le camping a adopté le format 'AAAA-MMM-xxx' composé des trois chaînes de caractères ci-dessous :
* 'AAAA' une année comprise entre 2018 et 2024 ;
* 'MMM' les trois premières lettres du mois en anglais ;
* 'xxx' désigne trois chiffres.

On décide d'écrire une fonction `facture_est_valide` pour tester si une chaîne de caractères représente un numéro de facture valide ou non. Voici quelques exemples du comportement attendu de la fonction facture est valide.

```
>>> facture_est_valide('2024-MAY-230')
True
>>> facture_est_valide('2012-MAY-230')
False
>>> facture_est_valide('2024-MAI-230')
False
>>> facture_est_valide('2024-JUN-23')
False
```

On considère le programme suivant :

```
calendrier = ['JAN', 'FEB', 'MAR', 'APR', 'MAY', 'JUN',
    'JUL', 'AUG', 'SEP', 'OCT', 'NOV', 'DEC']

def separe(chaine):
    """renvoie une liste constituée de chaînes qui étaient
    séparées par le caractère - """
    return chaine.split('-')

def que_des_chiffres(chaine):
    """renvoie vrai si chaine n'est constituée que
    des caractères de 0 à 9 faux sinon"""
    for car in chaine:
        if not(car in "012345789"):
            return False
    return True

def facture_est_valide(chaine):
    """renvoie vrai si chaine est une chaîne de
    caractères conforme au modèle de facture"""
    partie = separe(chaine)
    if not(len(partie) == 3):
        return False
    annee, mois, numero = partie[0], partie[1], partie[2]
    if not(que_des_chiffres(annee)):
        return False
    if not(len(annee) == 4) or not(2018 <= annee <= 2024):
        return False
    # Reste à faire vérifier les mois MMM
    pass
    # Reste à faire vérifier le numéro xxx *
    pass
    return True
```

On rappelle que la fonction split en Python divise une chaîne de caractères en une liste de sous-chaînes en fonction d'un séparateur spécifié.
Par exemple :

```
texte = 'Bonjour-le-monde'
separateur = '-'
resultat = texte.split(separateur)
```

donne comme résultat : `['Bonjour', 'le', 'monde']`

Question 2 – Avec le code actuel, une erreur se produit à l'exécution de la fonction `facture_est_valide` donnée ci-dessus.
Proposer une correction du code pour que cette erreur ne se produise plus.

Question 3 – Compléter le code afin de vérifier les mois et le numéro dans la fonction `facture_est_valide`. Pour chaque vérification, il est possible d'insérer une ou plusieurs lignes.

Appel 2	**Appeler le professeur pour lui présenter votre fonction et son fonctionnement ou en cas de difficultés.**

Question 4 – En vous aidant des 4 exemples du comportement attendu de la fonction `facture_est_valide` écrit plus haut, écrire un jeu de test **avec des assertions.**

Appel 3	**Appeler le professeur pour lui présenter les assertions ou en cas de difficultés.**

On souhaite écrire une fonction `chiffre_affaire` qui prend en paramètre la liste liste_facture constituée de dictionnaires, voir exemple ci-dessous et qui renvoie la somme des montants dûs pour toutes les factures du camping qui sont valides.

```
liste_facture = [ {'numero' : '2024-MAY-230', 'montant' : 145},
                  {'numero' : '2012-MAY-230', 'montant' : 230},
                 {'numero' : '2024-AMI-231', 'montant' : 55},
                  {'numero' : '2024-JUN-23', 'montant' : 16},
                  {'numero' : '2024-JUN-231', 'montant' : 10}]
```

Exemple : `chiffre_affaire(liste_facture)` renvoie `155`

```
>>> chiffre_affaire(liste_facture)
155
```

Question 5 – Compléter la fonction `chiffre_affaire` et tester votre fonction

```
def chiffre_affaire(liste):
    """ renvoie la somme des montants de la liste des factures en euros, passée en paramètre
    les factures non valides ne doivent pas être prises en compte dans le calcul de la somme.
    la liste de facture est une liste de dictionnaire.
    """
    pass
```

Appel 4	**Appeler le professeur pour lui présenter votre fonction et son fonctionnement ou en cas de difficultés.**